

Relatório Técnico: Implementação Modular NFA-ε → NFA → DFA

Pablo Belmiro

Este relatório descreve o processo de criação do meu conversor de autômatos, desenvolvido para o **Laboratório 01** de Teoria da Computação. A ideia desse relatório é explicar como saímos de um problema teórico e chegamos em um programa funcional em Haskell.

1. O Problema Proposto

O desafio era criar um programa que pegasse um **NFA-ε** (Autômato Finito Não-Determinístico com transições vazias) e o transformasse em um **DFA** (Autômato Finito Determinístico).

Como o Sipser menciona, o DFA é muito mais fácil para o computador "executar"/"rodar", mas o NFA com epsilons é muito mais fácil para projetarmos. O meu trabalho foi criar a ponte entre esses dois mundos.

2. Dados de Entrada (`nfae.yaml`)

Para alimentar o programa, usamos um arquivo YAML. Ele descreve o autômato *N₄* do livro:

- **Alfabeto:** `{a, b}`.
- **Estados:** `1, 2, 3`.
- **Estado Inicial:** `1`.
- **Estado Final:** `1` (apenas ele aceita a palavra).
- **Transições:** Temos transições comuns (ex: `1` com `b` vai para `2`) e a transição `epsilon` (ex: `1` com `epsilon` vai para `3`), que permite mudar de estado sem ler nenhuma letra.

3. O Código Main (`Main-2-1-conversao-automatos.hs`)

Para resolver o problema de forma didática, dividi o programa em duas grandes etapas:

Etapa 1: O Fecho-Epsilon e a Remoção do Epsilon

A primeira implementação foi da função `pegarFecho`. Ela é um **Cálculo de Ponto Fixo**: ela olha para um estado e valida algo como: "Até onde eu consigo chegar só seguindo setas vazias?". Ela repete (recursiva) isso até que não encontre nenhum estado novo.

Depois, construí a função `removerEpsilon`. Ela cria um novo autômato onde cada seta normal agora "embutiu" os saltos vazios.

- Se antes eu ia de `1 → 3` (vazio) e de `3  $\xrightarrow{a}$  1`, agora eu criei uma seta direta `1  $\xrightarrow{a}$  {1,3}`.

Etapa 2: A Construção de Subconjuntos

Com o autômato "limpo" de epsilons, usei a função `construcaoSubconjuntos`. Aqui, eu trato grupos de estados como um só.

- O programa começa no estado $\{1\}$.
- Ele valida algo como: "Se eu estiver no 1 e ler 'a', em quais estados eu posso terminar?". A resposta seria $\{1, 3\}$.
- Esse conjunto $\{1, 3\}$ vira um **novo estado** no meu DFA.
- O processo se repete (usando a função `resolver`) até que todos os caminhos possíveis sejam mapeados.

4. O Resultado Final (`dfa.yaml`)

O arquivo gerado (`dfa.yaml`) pode parecer grande, mas possui sentido técnico:

- **6 Estados Encontrados:** O programa descobriu que existem 6 combinações diferentes de estados em que o autômato pode "estar" ao mesmo tempo (ex: $\{1,2,3\}$, $\{2,3\}$, $\{1,3\}$, etc).
- **Determinismo:** Se observarmos, cada estado tem exatamente uma saída para `a` e uma para `b`, fechando as possibilidades exatas das saídas.
- **Herança de Aceitação:** Todo estado no DFA que "dentro de si" tenha o estado `1` original foi marcado como final. Por isso, estados como $\{1,3\}$ e $\{1,2,3\}$ aparecem nos `final_states`.

5. Conclusão

O código prova que cada transformação é uma operação independente sobre o modelo formal do autômato, facilitando a verificação de corretude em cada estágio.

Valores Gerados:

- **Iniciais:** O estado inicial do DFA é o conjunto `{initial_state}` do NFA limpo.
- **Finais:** Aplicamos o critério de interseção ($R \cap F_{NFA} \neq \emptyset$) para definir os estados de aceitação do DFA.

Aluno: Pablo Belmiro

Disciplina: Teoria da Computação

Professor: Dr. Jefferson O. Andrade